Experiment No. 2

Experiment Title	To perform Data Preprocessing using Python
Course Outcomes	CO2: Organize and prepare the data needed for data mining using preprocessing techniques.
Date of Preparation	
Date of Submission	
Signature & Grade	

1. Aim

To perform data preprocessing using Python (pandas, NumPy and scikit-learn) in Google Colab: cleaning, outlier treatment, data reduction, encoding and scaling, on an airline passenger dataset.

2. Theory

Raw data collected from real systems is often incomplete, inconsistent and on very different scales. Data preprocessing converts it into a clean, consistent form so that data mining and machine learning methods give reliable results. The main steps used in this experiment are:

Step	Purpose	Method used
Data cleaning	Remove duplicate records and useless columns, and fill missing values.	drop_duplicates(), drop(), median / mode fill
Outlier treatment	Limit extreme values that distort statistics.	IQR method, values capped at the bounds
Data reduction	Remove columns that repeat the same information.	Correlation matrix, threshold 0.9
Data transformation	Convert text to numbers and bring all columns to the same scale.	Ordinal and one-hot encoding, Min-Max scaling
Train-test split	Keep unseen data aside to test a model fairly.	train_test_split (80% / 20%)

IQR method: IQR = Q3 − Q1. A value below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR is treated as an outlier. Min-Max scaling: x' = (x − min) / (max − min), which maps every value into the range 0 to 1.

3. Dataset and Tools

Dataset: Airline Passenger Satisfaction (Kaggle), file train.csv, with 103,904 records and 25 columns. It holds passenger details, flight details and service ratings (0–5). The column 'satisfaction' says whether the passenger was 'satisfied' or 'neutral or dissatisfied'.

Source: kaggle.com/datasets/teejmahal20/airline-passenger-satisfaction

Tools: Python 3 in Google Colab with the libraries pandas, NumPy, Matplotlib, Seaborn and scikit-learn.

4. Implementation

4.1 Load the Dataset

```
df = pd.read_csv('train.csv')
print('Rows, Columns :', df.shape)
df.head()
```

```
Rows, Columns : (103904, 25)
```

	Unnamed: 0	id	Gender	Customer Type	Age	Type of Travel	Class	Flight Distance	Inflight wifi service	Departure/Arrival time convenient	Ease of Online booking	Gate location	Food and drink	bo
0	0	70172	Male	Loyal Customer	13	Personal Travel	Eco Plus	460	3	4	3	1	5	
1	1	5047	Male	disloyal Customer	25	Business travel	Business	235	3	2	3	3	1	
2	2	110028	Female	Loyal Customer	26	Business travel	Business	1142	2	2	2	2	5	
3	3	24026	Female	Loyal Customer	25	Business travel	Business	562	2	5	5	5	2	
4	4	119299	Male	Loyal Customer	61	Business travel	Business	214	3	3	3	3	4	

Figure 1: Dataset loaded — 103,904 rows and 25 columns

4.2 Check Missing Values

```
missing = df.isnull().sum()
missing[missing > 0]          # columns that contain missing values
```

```
Columns that contain missing values:
Missing Count  Missing %
No missing values found.
```

Figure 2: Result of the missing-value check

4.3 Handle Missing Values

Numeric columns with missing values are filled with the median, because the delay columns are heavily right-skewed and a few very large delays would pull the mean upwards. Text columns would be filled with the most frequent value (mode).

```
for col in df.select_dtypes(include='number').columns:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].median())
for col in df.select_dtypes(exclude='number').columns:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].mode()[0])
```

```
Missing values BEFORE: 0
Missing values AFTER : 0
```

Figure 3: Missing values before and after the fill step

4.4 Remove Duplicates and Irrelevant Columns

No duplicate rows were found, so all 103,904 rows were kept. The columns 'Unnamed: 0' (a row index) and 'id' (an identifier) say nothing about satisfaction and were dropped, which reduced the data from 25 to 23 columns.

```
df = df.drop_duplicates()
df = df.drop(columns=['Unnamed: 0', 'id'])
```

4.5 Outlier Detection and Treatment

The IQR method was applied to Flight Distance, Departure Delay in Minutes and Arrival Delay in Minutes. Values outside the bounds were capped (not deleted), so every row is kept.

Column	Q1	Q3	Lower bound	Upper bound
Flight Distance	414.0	1743.0	−1579.5	3736.5
Departure Delay in Minutes	0.0	12.0	−18.0	30.0
Arrival Delay in Minutes	0.0	13.0	−19.5	32.5

```
for col in ['Flight Distance', 'Departure Delay in Minutes', 'Arrival Delay in Minutes']:
    Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
    IQR = Q3 - Q1
    df[col] = df[col].clip(lower=Q1 - 1.5*IQR, upper=Q3 + 1.5*IQR)
```

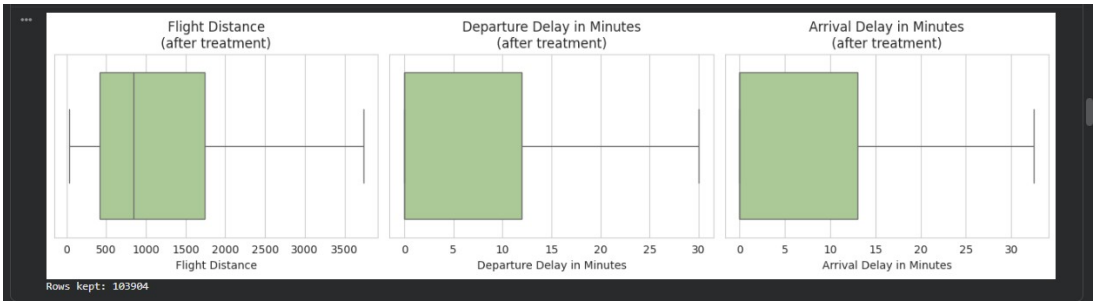


Figure 4: Box plots after outlier treatment — all values lie within the IQR bounds; 103,904 rows kept

4.6 Data Reduction (Correlation)

A correlation matrix of the numeric columns was drawn. If two columns had a correlation above 0.9, one of them would be dropped. The two delay columns show the strongest relationship, but it stays below 0.9, so no column was dropped and 23 columns remained.

```
corr = df.select_dtypes(include='number').corr()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
to_drop = [c for c in upper.columns if (upper[c].abs() > 0.9).any()]
df = df.drop(columns=to_drop)
```

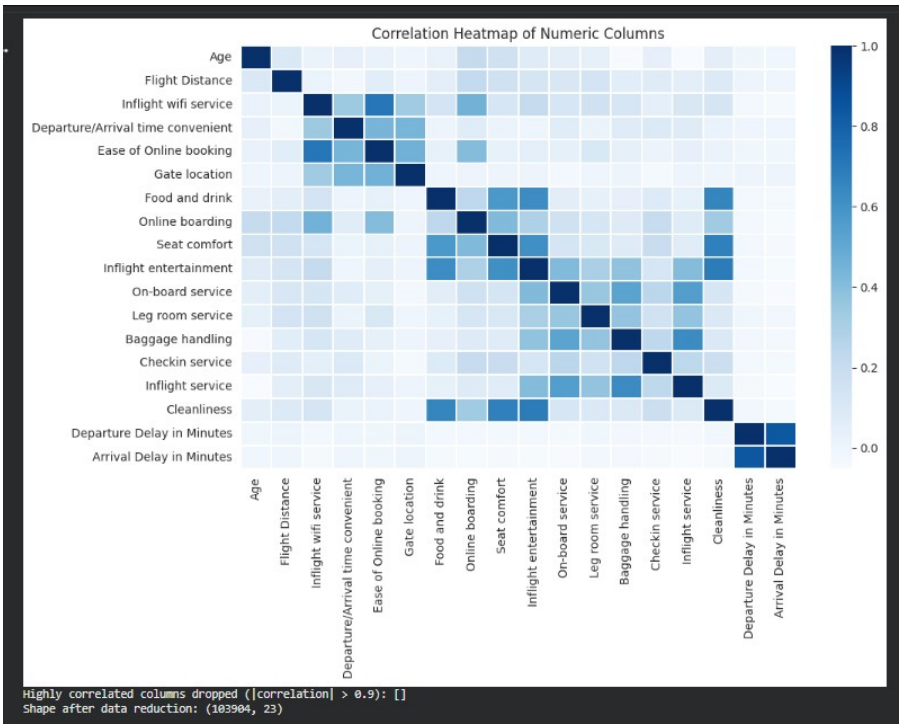
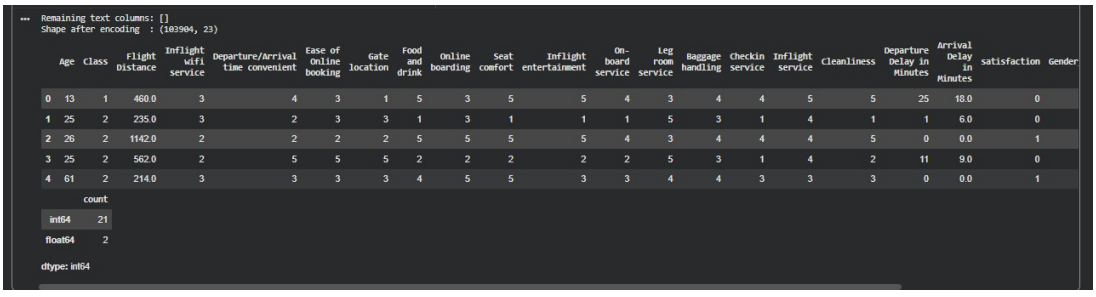


Figure 5: Correlation heatmap of the numeric columns; no column exceeded the 0.9 threshold

4.7 Encode Categorical Columns

- Class has a natural order, so it was ordinal encoded: Eco = 0, Eco Plus = 1, Business = 2.
- The target 'satisfaction' was mapped to numbers: neutral or dissatisfied = 0, satisfied = 1.
- Gender, Customer Type and Type of Travel have no order, so they were one-hot encoded (drop_first = True).

```
df['satisfaction'] = df['satisfaction'].map({'neutral or dissatisfied': 0, 'satisfied': 1})
df['Class'] = df['Class'].map({'Eco': 0, 'Eco Plus': 1, 'Business': 2})
df = pd.get_dummies(df, columns=['Gender', 'Customer Type', 'Type of Travel'],
                    drop_first=True, dtype=int)
```



Remaining text columns: []
Shape after encoding : (103904, 23)

	Age	Class	Flight Distance	Inflight wifi service	Departure/Arrival time convenient	Ease of Online booking	Gate location	Food and drink	Online boarding	Seat comfort	Inflight entertainment	On-board service	Leg room service	Baggage handling	Checkin service	Inflight service	Cleanliness	Departure Delay in Minutes	Arrival Delay in Minutes	satisfaction	Gender
0	13	1	460.0	3	4	3	1	5	3	5	5	4	3	4	4	5	5	25	18.0	0	
1	25	2	235.0	3	2	3	3	1	3	1	1	1	5	3	1	4	1	1	6.0	0	
2	26	2	1142.0	2	2	2	2	5	5	5	5	4	3	4	4	4	5	0	0.0	1	
3	25	2	562.0	2	5	5	5	2	2	2	2	2	5	3	1	4	2	11	9.0	0	
4	61	2	214.0	3	3	3	3	4	5	5	3	3	4	4	3	3	3	0	0.0	1	

	count
int64	21
float64	2

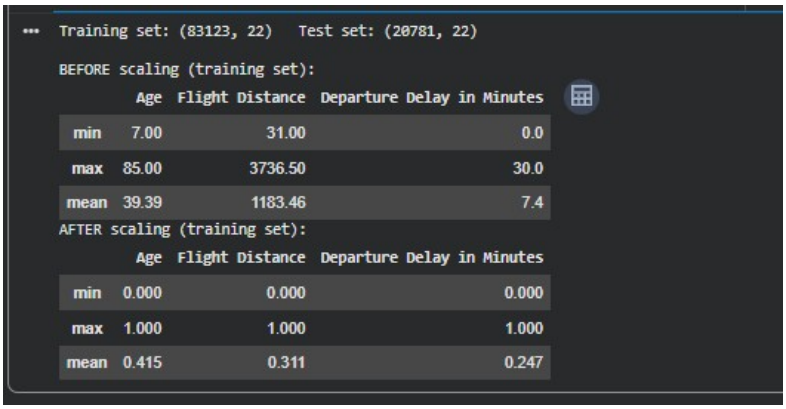
dtype: int64

Figure 6: After encoding — no text columns remain (23 columns; 21 integer and 2 float)

4.8 Train-Test Split and Feature Scaling

The data was split 80% / 20% (stratified, so both parts keep the same class balance). The Min-Max scaler learns the minimum and maximum from the training data only and is then applied to both parts, so no information from the test data leaks into the training process.

```
X = df.drop(columns=['satisfaction']);  y = df['satisfaction']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```



Training set: (83123, 22) Test set: (20781, 22)

BEFORE scaling (training set):

	Age	Flight Distance	Departure Delay in Minutes
min	7.00	31.00	0.0
max	85.00	3736.50	30.0
mean	39.39	1183.46	7.4

AFTER scaling (training set):

	Age	Flight Distance	Departure Delay in Minutes
min	0.000	0.000	0.000
max	1.000	1.000	1.000
mean	0.415	0.311	0.247

Figure 7: Training and test set sizes, and values before and after scaling

The training set has 83,123 records and the test set 20,781, each with 22 features. After scaling, every column has a minimum of 0 and a maximum of 1.

5. Summary of Results

Step	Result
Original dataset	103,904 rows × 25 columns
Duplicate rows removed	0 (all rows kept)
Irrelevant columns dropped	2 (Unnamed: 0, id) → 23 columns
Missing values after the fill step	0
Outlier treatment	3 columns capped using the IQR bounds; 103,904 rows kept
Columns dropped for correlation above 0.9	None
Text columns after encoding	0 (all columns numeric)
Train / test split	83,123 / 20,781 records, 22 features
Scaling	All features in the range 0 to 1

6. Conclusion

Data preprocessing was performed on the Airline Passenger Satisfaction dataset using Python in Google Colab. The data was loaded and checked for missing and duplicate values, irrelevant columns were removed, and outliers in Flight Distance and the two delay columns were capped using the IQR method without losing any row. A correlation check found no pair of columns above the 0.9 threshold. The text columns were converted to numbers using ordinal and one-hot encoding, the data was split into 83,123 training and 20,781 test records, and all features were scaled to the range 0 to 1 using Min-Max scaling fitted on the training data only. The resulting dataset is fully numeric, consistent and ready for data mining tasks such as classification and clustering.